SALUS SECURITY

JUL 2026

CODE SECURITY ASSESSMENT

TERMIX AACP

Overview

Project Summary

- Name: Termix AACP
- Platform: EVM-compatible chains
- Language: Solidity
- Repository:
    - https://github.com/TermiX-official/termix-aacp-contract
- Audit Range: See Appendix - 1

Project Dashboard

Application Summary

Name	Termix AACP
Version	v2
Type	Solidity
Dates	Jul 20 2026
Logs	Jul 09 2026, Jul 20 2026

Vulnerability Summary

Total High-Severity issues	4
Total Medium-Severity issues	7
Total Low-Severity issues	3
Total informational issues	3
Total	17

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (https://t.me/salusec), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Providers will lose payment if brand refunds slots	High	Business Logic	Resolved
2	Anyone can open slot challenge to block payment for providers	High	Business Logic	Resolved
3	Challenge evaluator list can be manipulated	High	Business Logic	Resolved
4	Missing challenge fee and escalation fee mechanism	High	Business Logic	Acknowledged
5	Users' assets may be stuck in CampaignVault	Medium	Business Logic	Acknowledged
6	Improper reputation record	Medium	Business Logic	Resolved
7	Improper reputation score calculation	Medium	Business Logic	Resolved
8	Challenge results may be manipulated	Medium	Business Logic	Acknowledged
9	Providers' available assets may be locked temporary	Medium	Business Logic	Resolved
10	Providers may lose opportunity to open one challenge	Medium	Business Logic	Resolved
11	Centralization risk	Medium	Centralization	Acknowledged
12	Slot settle may be blocked if the recipient is in the blacklist	Low	Business Logic	Acknowledged
13	Inconsistent total fee bps limitation	Low	Business Logic	Resolved
14	Users can block fundCampaign via frontrun	Low	Business Logic	Acknowledged
15	Inconsistent fee limit setting in different contracts	Informational	Business Logic	Resolved
16	Missing event emission in setVerdictTimeout	Informational	Business Logic	Resolved
17	Redundant code	Informational	Business Logic	Acknowledged

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Providers will lose payment if brand refunds slots

Severity: High	Category: Business Logic
Target: - contracts/CampaignVault.sol	

Description

In CampaignVault, anyone can become one campaign's brand. The brand will release one slot for one provider to pay for the task, and the brand will refund slots if the provider's task is rejected or expired.

The problem here is that anyone can become the campaign manager. Even if the provider finishes the expected task, the campaign's brand can also refund all slots. This will cause providers to lose their expected payment.

contracts/CampaignVault.sol:L255-L265

```
function refundSlot(bytes32 campaignId, bytes32 slotId) external nonReentrant {
    Campaign storage c = campaigns[campaignId];
    if (msg.sender != c.brand) revert NotBrand();
    if (c.state != CampaignState.Funded) revert InvalidState();
    if (slotStates[campaignId][slotId] != SlotState.None) revert InvalidState();
    if (c.committedSlots >= c.totalSlots) revert CapacityExhausted();
    slotStates[campaignId][slotId] = SlotState.Refunded;
    c.committedSlots += 1;
    settlementToken.safeTransfer(c.brand, c.rewardPerSlot);
    emit SlotRefunded(campaignId, slotId, c.rewardPerSlot);
}
```

Recommendation

If the brand wants to refund the rejected/expired slot, the related provider should have the permission to open one challenge before this refund.

Status

This issue has been resolved by the team with commit 1dbcff4.

2. Anyone can open slot challenge to block payment for providers

Severity: High	Category: Business Logic
Target: - contracts/CampaignVault.sol	

Description

In CampaignVault, the participant(brand/provider) can open one slot challenge.

The problem here is that the provider verification does not work as expected. Anyone can trigger this function with `provider = msg.sender`. The challenge will pass no matter if this msg.sender is one real provider or not. When the `committedSlots` is increased to totalSlots, the normal payment path releaseSlot/refundSlot cannot work as expected.

contracts/CampaignVault.sol:L279-L291

```
function openSlotChallenge(
    bytes32 campaignId,
    bytes32 slotId,
    address provider,
    uint256[3] calldata evaluatorAgentIds
) external nonReentrant {
    Campaign storage c = campaigns[campaignId];
    if (c.state != CampaignState.Funded) revert InvalidState();
    if (provider == address(0)) revert ZeroAddress();
    if (msg.sender != provider && msg.sender != c.brand) revert NotParticipant();
}
```

Recommendation

The brand should add the allowed provider for each slot.

Status

This issue has been resolved by the team with commit 1dbcff4.

3. Challenge evaluator list can be manipulated

Severity: High	Category: Business Logic
Target: - contracts/CampaignVault.sol	

Description

In CampaignVault, the participant(brand/provider) can open one slot challenge. The brand/provider can assign the evaluator for this challenge. The slot id is global.

In function `openSlotChallenge`, the `slotStates[campaignId][slotId]` is used to prevent one slot from being challenged multiple times. The problem here is that users can challenge the same slot multiple times via different campaigns. This will cause that anyone with one campaign can open or update one slot's challenge.

contracts/CampaignVault.sol:L305-L310

```
function openSlotChallenge(
    bytes32 campaignId,
    bytes32 slotId,
    address provider,
    uint256[3] calldata evaluatorAgentIds
) external nonReentrant {
    if (slotStates[campaignId][slotId] != SlotState.None) revert InvalidState();
    slotStates[campaignId][slotId] = SlotState.Challenged;
    c.committedSlots += 1;
    slotDisputeSubject[slotId] = SlotDisputeSubject({campaignId: campaignId, slotId:
slotId, provider: provider});
    DisputeLib.setPanel(slotDisputes[slotId], [evaluatorAgentIds[0],
evaluatorAgentIds[1], evaluatorAgentIds[2]]);
}
```

Recommendation

Verify whether this slot Id is challenged via `slotDisputeSubject`.

Status

This issue has been resolved by the team with commit 1dbcff4.

4. Missing challenge fee and escalation fee mechanism

Severity: High	Category: Business Logic

Target:
- contracts/CampaignVault.sol

Description

In CampaignVault, the participant(brand/provider) can open one slot challenge. And they can escalate the challenge result via the function arbitrate.

The problem here is that users do not need to pay any fees for the challenge and the arbitrate. The evaluator fee and the arbitrator fee will be charged from the original fund which should belong to the provider or the original brand. The loser side(brand/provider) can always trigger a challenge and arbitrate to deduct the winner's profit.

For example, the assigned provider knows he cannot submit one proper task, but this provider can always trigger a challenge and arbitrate to force the brand paying for these evaluator fees and escalation fee.

contracts/CampaignVault.sol:L349-L358

```
function escalateToArbitration(bytes32 slotId, uint256 arbitratorAgentId) external
nonReentrant {
    DisputeLib.Dispute storage d = slotDisputes[slotId];
    if (d.phase != DisputeLib.DisputePhase.AwaitingDecision) revert InvalidState();
    SlotDisputeSubject storage s = slotDisputeSubject[slotId];
    if (msg.sender != s.provider && msg.sender != campaigns[s.campaignId].brand) revert
NotParticipant();
    if (arbitratorAgentId == 0 ||
!IDisputeRegistry(disputeRegistry).isArbitratorAgent(arbitratorAgentId)) revert
NotArbitrator();
    if (DisputeLib.inPanel(d, arbitratorAgentId)) revert InvalidParams();
    DisputeLib.markEscalated(d, arbitratorAgentId);
    emit SlotDisputeEscalated(slotId, arbitratorAgentId);
}
```

Recommendation

Charge some fees for the originator.

Status

This issue has been acknowledged by the team. The team stated that this behavior is aligned with the current business requirement, where providers are not required to post additional penalty bonds for CampaignVault disputes. Therefore, the team acknowledged the issue and deferred a challenge/escalation fee mechanism to a future dispute-incentive redesign.

5. Users' assets may be stuck in CampaignVault

Severity: Medium	Category: Business Logic
Target: - contracts/CampaignVault.sol	

Description

In CampaignVault, the participant(brand/provider) can open one slot challenge. The evaluators should vote for this challenge slot.

The problem here is that if one evaluator does not vote for some reason, this challenge may fail to reach the agreement. The assets will be stuck in this contract.

contracts/CampaignVault.sol:L319-L331

```
function castVote(bytes32 slotId, uint256 evaluatorAgentId, bool providerUpheld)
external nonReentrant {
    DisputeLib.Dispute storage d = slotDisputes[slotId];
    if (d.phase != DisputeLib.DisputePhase.Voting) revert InvalidState();
    // Only this evalator can vote.
    if (agentNFT.ownerOf(evaluatorAgentId) != msg.sender) revert NotEvaluator();
    (bool reached, bool upheld) = DisputeLib.recordVote(d, evaluatorAgentId,
providerUpheld);
    emit SlotEvaluatorVoteCast(slotId, evaluatorAgentId, providerUpheld);
    if (reached) {
        uint64 deadline = uint64(block.timestamp + verdictTimeout);
        verdictDeadlineAt[slotId] = deadline;
        emit SlotEvaluatorVerdictReached(slotId, upheld, deadline);
    }
}
```

Recommendation

Refer to the TermixEscrow contract, add one emergent challenge settle function.

Status

This issue has been acknowledged by the team. The team stated that the evaluator set is currently fully managed by the platform and consists of three trusted evaluators, so the risk of evaluators not voting is considered acceptable in the current deployment. The team acknowledged that a voting-timeout or evaluator-penalty mechanism will be needed before opening the evaluator set more broadly.

6. Improper reputation record

Severity: Medium	Category: Business Logic
Target: - contracts/TermixReputation.sol	

Description

In the TermixReputation contract, the providers' performance will be recorded and scored. Then the clients can choose the expected providers based on their score.

The problem here is that in one order, the `disputedOrders` may be added twice. This will decrease this provider's performance order.

contracts/Reputation.sol:L349-L358

```
function recordOrderResult(uint256 agentId, bool success, bool disputed) external {
    if (!recorders[msg.sender]) revert NotRecorder();
    AgentStats storage item = stats[agentId];
    item.completedOrders += 1;
    if (success) item.successfulOrders += 1;
    if (disputed) item.disputedOrders += 1;
    item.lastUpdatedAt = block.timestamp;
    emit OrderResultRecorded(agentId, success, disputed, getScore(agentId));
}

function recordChallengeResult(uint256 agentId, bool providerUpheld) external {
    if (!recorders[msg.sender]) revert NotRecorder();
    AgentStats storage item = stats[agentId];
    if (!providerUpheld) item.disputedOrders += 1;
    item.lastUpdatedAt = block.timestamp;
    emit OrderResultRecorded(agentId, providerUpheld, true, getScore(agentId));
}
```

Recommendation

If one order is challenged and this is disputed with providerUpheld, the `disputedOrders` should not be added.

Status

This issue has been resolved by the team with commit 1dbcff4.

7. Improper reputation score calculation

Severity: Medium	Category: Business Logic
Target: - contracts/TermixReputation.sol	

Description

In the TermixReputation contract, the providers' performance will be recorded and scored. Then the clients can choose the expected providers based on their score.

The function `getScore` is used to calculate the provider's score. The formula is as follows:

`sucessfulOrders * 100 / completedOrders - disputedOrders * 5`.

The `sucessfulOrder * 100 / completedOrders` 's maximum score is 100. This means that if this provider has 20 disputed orders, the score will always be 1 no matter how many successful orders they have submitted. This will cause that the providers with higher successful order rate have one low score. They will have less chance to be picked.

contracts/TermixReputation.sol:L349-L358

```
function getScore(uint256 agentId) public view returns (uint8) {
    AgentStats storage item = stats[agentId];
    if (item.completedOrders == 0) return 50;
    uint256 successScore = (item.successfulOrders * 100) / item.completedOrders;
    uint256 disputePenalty = item.disputedOrders * 5;
    if (successScore <= disputePenalty) return 1;
    uint256 score = successScore - disputePenalty;
    if (score > 100) return 100;
    if (score == 0) return 1;
    return uint8(score);
}
```

Recommendation

Refactor the logic for the function getScore.

Status

This issue has been resolved by the team with commit 1dbcff4.

8. Challenge results may be manipulated

Severity: Medium	Category: Business Logic
Target: - contracts/TermixEscrow.sol	

Description

In the TermixEscrow contract, the participants in one order can choose to open one challenge. In function openChallenge, the participants can assign any valid evaluators from the whitelist. If this participant takes control of one or two evaluators, the participants are easy to manipulate the challenge result.

contracts/TermixEscrow.sol:L309-L345

```
function openChallenge(bytes32 orderId, uint256[3] calldata evaluatorAgentIds) external
nonReentrant {
    for (uint256 i = 0; i < EVALUATOR_PANEL_SIZE; i++) {
        uint256 agentId = evaluatorAgentIds[i];
        if (agentId == 0 || !isEvaluatorAgent[agentId]) revert NotEvaluator();
        for (uint256 j = 0; j < i; j++) {
            // donot allow the dup agent
            if (evaluatorAgentIds[j] == agentId) revert InvalidParams();
        }
        d.evaluatorAgentIds[i] = agentId;
    }
}
```

Recommendation

Consider choosing the evaluator via random number.

Status

This issue has been acknowledged by the team. The team stated that the evaluator set is currently limited to three platform-managed evaluators, so participant-selected panels do not introduce a practical manipulation risk in the current deployment. The team acknowledged that evaluator selection and evaluator-penalty mechanisms should be redesigned before expanding the evaluator set.

9. Providers' available assets may be locked temporary

Severity: Medium	Category: Business Logic
Target: - contracts/TermixEscrow.sol	

Description

In the TermixEscrow contract, the client can create one order with the assigned provider. The order will lock the provider's deposit assets.

The problem here is that malicious users can frontrun to create some orders with difficult missions and short deadlines. This will cause normal providers' available assets to be locked and they can not participate in normal orders. Once this short deadline passes, the client can refund assets via the cancelExpired.

contracts/TermixEscrow.sol:L235-L267

```
function createOrder(
    uint256 providerAgentId,
    bytes32 agreementHash,
    uint256 budget,
    uint256 deadline,
    uint8 settlementMode,
    uint256 challengeWindow
) external nonReentrant returns (bytes32 orderId) {
    if (address(staking) != address(0)) {
        staking.lockForOrder(orderId, providerAgentId, budget);
    }
}
```

Recommendation

From the view of the provider, it will be better to let the provider confirm this order and then lock their assets.

Status

This issue has been resolved by the team with commit 1dbcff4.

10. Providers may lose opportunity to open one challenge

Severity: Medium	Category: Business Logic
Target: - contracts/TermixEscrow.sol	

Description

In the TermixEscrow contract, the client can create one order with the assigned provider. After the provider submits the delivery, and the client does not accept the delivery for a while, the provider can open one challenge for this payment.

The problem here is that if the client sets this challengeWindow to one quite short time slot. When users submit the delivery near to the deadline, there is one short time slot for users to open one challenge. If the provider misses this narrow challenge window, the provider will miss this payment if the client does not accept this delivery.

contracts/TermixEscrow.sol:L249-L260

```
function createOrder(
    uint256 providerAgentId,
    bytes32 agreementHash,
    uint256 budget,
    uint256 deadline,
    uint8 settlementMode,
    uint256 challengeWindow
) external nonReentrant returns (bytes32 orderId) {
    orderId = keccak256(abi.encodePacked(address(this), msg.sender, providerAgentId,
agreementHash, nonce++));
    Order storage order = orders[orderId];
    order.client = msg.sender;
    order.providerAgentId = providerAgentId;
    order.agreementHash = agreementHash;
    order.budget = budget;
    order.deadline = deadline;
    order.settlementMode = settlementMode;
    order.challengeWindowEndsAt = deadline + challengeWindow;
}
```

Recommendation

Add one minimum time limit for the challenge window.

Status

This issue has been resolved by the team with commit 1dbcff4.

11. Centralization risk

Severity: Medium	Category: Business Logic
Target: - contracts/TermixEscrow.sol - contracts/CampaignVault.sol	

Description

In the TermixEscrow/CampaignVault contract, there exists some privileged roles, `AdminRole`, `OperatorRole`, `UpgraderRole`. These roles have the authority to execute some key functions such as `upgradeTo`, `setDependencies`, etc.

If these roles' private keys are compromised, an attacker could trigger these functions to withdraw all funds.

contracts/TermixEscrow.sol:L180-L188

```
function setDependencies(address staking_, address reputation_, address feeRecipient_)
external onlyRole(OPERATOR_ROLE) {
    if (feeRecipient_ == address(0)) revert ZeroAddress();

    staking = ITermixStaking(staking_);
    reputation = ITermixReputation(reputation_);
    feeRecipient = feeRecipient_;
    emit DependenciesSet(staking_, reputation_, feeRecipient_);
}
```

contracts/TermixEscrow.sol:L612-L613

```
   function _authorizeUpgrade(address newImplementation) internal override
onlyRole(UPGRADER_ROLE) {}
```

Recommendation

We recommend transferring privileged accounts to multi-sig accounts with timelock governors for enhanced security. This ensures that no single person has full control over the accounts and that any changes must be authorized by multiple parties.

Status

This issue has been acknowledged by the team.

12. Slot settle may be blocked if the recipient is in the blacklist

Severity: Low	Category: Business Logic

Target:
- contracts/CampaignVault.sol

Description

In the CampaignVault contract, when users accept the evaluator's result, this dispute will be settled. Evaluator fees will be transferred to the evaluators.

The problem here is that if this evaluator is in the blacklist of settlement tokens, this whole transaction will be reverted. The whole payment will be stuck in this contract.

contracts/CampaignVault.sol:L249-L260

```
function _settleSlotDispute(bytes32 slotId, bool providerUpheld, uint256
arbitratorAgentId) internal {
    uint256 paidEvaluators = 0;
    if (evaluatorFee > 0) {
        uint256[3] memory amounts = DisputeLib.splitEvaluatorFee(evaluatorFee);
        for (uint256 i = 0; i < 3; i++) {
            if (amounts[i] == 0) continue;
            address recipient = agentNFT.ownerOf(d.evaluatorAgentIds[i]);
            settlementToken.safeTransfer(recipient, amounts[i]);
            paidEvaluators += amounts[i];
            emit SlotDisputeFeePaid(slotId, recipient, d.evaluatorAgentIds[i],
amounts[i], false);
        }
    }
}
```

Recommendation

Use a pull-over-push pattern for evaluator/arbitrator fee transfer.

Status

This issue has been acknowledged by the team.

13. Inconsistent total fee bps limitation

Severity: Low	Category: Business Logic
Target: - contracts/TermixEscrow.sol	

Description

In the TermixEscrow contract, the operator role can set the dispute fees via the function `setDisputeFees`. There is one hard fee limit, the total fees cannot exceed 50%.

The problem here is that when the operator sets the protocol fee via the function `setProtocolFeeBps`, there is not the same fee limit. This will cause the total fee to exceed 50%.

contracts/TermixEscrow.sol:L249-L260

```
function setDisputeFees(uint256 evaluatorFeeBps_, uint256 arbitratorFeeBps_) external
onlyRole(OPERATOR_ROLE) {
    if (evaluatorFeeBps_ + arbitratorFeeBps_ + protocolFeeBps > 5_000) revert
FeeTooHigh();
    evaluatorFeeBps = evaluatorFeeBps_;
    arbitratorFeeBps = arbitratorFeeBps_;
    emit DisputeFeesSet(evaluatorFeeBps_, arbitratorFeeBps_);
}
```

Recommendation

Add the same fee check in the function `setProtocolFeeBps`.

Status

This issue has been resolved by the team with commit 1dbcff4.

14. Users can block fundCampaign via frontrun

Severity: Low	Category: Business Logic
Target: - contracts/CampaignVault.sol	

Description

In the CampaignVault contract, the brand can fund one campaign via the function `fundCampaign`.

The problem here is that malicious users can frontrun this campaign with the same campaignId, and dust rewardPerSlot. Malicious users don't need to pay any reward here to block normal users' fund campaigns.

contracts/CampaignVault.sol:L213-L224

```
function fundCampaign(bytes32 campaignId, uint256 rewardPerSlot, uint32 totalSlots)
external nonReentrant {
    if (campaignId == bytes32(0) || rewardPerSlot == 0 || totalSlots == 0) revert
InvalidParams();
    Campaign storage c = campaigns[campaignId];
    if (c.state != CampaignState.None) revert AlreadyExists();
    uint256 totalStaked = rewardPerSlot * uint256(totalSlots);
    c.brand = msg.sender;
    c.rewardPerSlot = rewardPerSlot;
    c.totalSlots = totalSlots;
    c.state = CampaignState.Funded;
    settlementToken.safeTransferFrom(msg.sender, address(this), totalStaked);
    emit CampaignFunded(campaignId, msg.sender, rewardPerSlot, totalSlots, totalStaked);
}
```

Recommendation

Bind the campaign storage key to the brand address.

Status

This issue has been acknowledged by the team.

2.3 Informational Findings

15. Inconsistent fee limit setting in different contracts

Severity: Informational	Category: Redundancy

Target:
- contracts/CampaignVault.sol
- contracts/TermixEscrow.sol

Description

In CampaignVault/TermixEscrow, the operator role can set the fee bps. Both functions have a similar maximum fee check.

The problem here is that different contracts use different fee hardcap. This will cause this inconsistent behavior from different contracts.

contracts/CampaignVault.sol:L213-L224

```
function setDisputeDeps(
    address agentNFT_,
    address disputeRegistry_,
    uint256 evaluatorFeeBps_,
    uint256 arbitratorFeeBps_,
    uint256 verdictTimeout_
) external onlyRole(OPERATOR_ROLE) {
    if (agentNFT_ == address(0) || disputeRegistry_ == address(0)) revert ZeroAddress();
       // But in CampaignVault, the whole fee bps should be less than 10000.
    if (protocolFeeBps + evaluatorFeeBps_ + arbitratorFeeBps_ > 10_000) revert
FeeTooHigh();
    ...
}
```

contracts/TermixEscrow.sol:L216-L222

```
function setDisputeFees(uint256 evaluatorFeeBps_, uint256 arbitratorFeeBps_) external
onlyRole(OPERATOR_ROLE) {
    if (evaluatorFeeBps_ + arbitratorFeeBps_ + protocolFeeBps > 5_000) revert
FeeTooHigh();
    evaluatorFeeBps = evaluatorFeeBps_;
    arbitratorFeeBps = arbitratorFeeBps_;
    emit DisputeFeesSet(evaluatorFeeBps_, arbitratorFeeBps_);
}
```

Recommendation

Use the same fee cap to make the behavior consistent.

Status

This issue has been resolved by the team with commit 1dbcff4.

16. Missing event emission in setVerdictTimeout

Severity: Informational	Category: Redundancy
Target: - contracts/TermixEscrow.sol	

Description

In the TermixEscrow contract, the operator role can set the verdict timeout via the function `setVerdictTimeout`.

The problem here is that the function misses the update event emission.

contracts/TermixEscrow.sol:L225-L228

```
function setVerdictTimeout(uint256 verdictTimeout_) external onlyRole(OPERATOR_ROLE) {
    verdictTimeout = verdictTimeout_;
    // @audit-info suggest to emit the event to key parameter's change.
}
```

Recommendation

Emit events for key parameter update.

Status

This issue has been resolved by the team with commit 1dbcff4.

17. Redundant code

Severity: Informational	Category: Redundancy
Target: - contracts/TermixEscrow.sol	

Description

In the TermixEscrow contract, when the client creates one order, the client will assign one settlement mode.

The problem here is that this settlement mode is not used in the whole contract.

contracts/TermixEscrow.sol:L235-L257

```
function createOrder(
    uint256 providerAgentId,
    bytes32 agreementHash,
    uint256 budget,
    uint256 deadline,
    uint8 settlementMode,
    uint256 challengeWindow
) external nonReentrant returns (bytes32 orderId) {
    order.settlementMode = settlementMode;
}
```

Recommendation

Remove the redundant code.

Status

This issue has been acknowledged by the team.

Appendix

Appendix 1 - Files in Scope

This audit covered the following files in commit 0767d27:

File	SHA-1 hash
DisputeLib.sol	e536b36f654ff77298d3e475c84ebcf3b7da9688
TermixStaking.sol	c7c58199c8bfbb36d0fe98ac359ee0c85bfecad1
TermixReputation.sol	ff5f82d208674e9cc5205e3c19df0f6ec319035d
TermixEscrow.sol	f339a7f91d76c60eccaf1c5b4a826c22ff81fbd7
TermixAgentNFT.sol	a96e256c0335a8c9fde6fc3afa586e4b9a42447b
CampaignVault.sol	4eca05861c37b0fe5211fdc2bb90822412655298

The fixed version reviewed covered the following files in commit cd3ebd8:

File	SHA-1 hash
DisputeLib.sol	7608f5386a5625e1252f636995ce1b04754308f9
TermixStaking.sol	e7b23be4b694a38ed051a4a203be288ca224b0e5
TermixReputation.sol	6542e938871d501a047200346ce4c08db9ec98c1
TermixEscrow.sol	5949471d49d6348579963880085 3916f70ee625b
TermixAgentNFT.sol	d496b47cef7e7a667c7dcda4606f50e598cc67ce
CampaignVault.sol	76b82db447c8b7f96be6edef318eda98717a48ae